FDTD Solver for Maxwell's Equations

The Yee Algorithm

2D Electromagnetic Simulation in TM Mode

2D TM-Mode Implementation with Performance Analysis

Explicit Loop vs. NumPy Vectorized Solvers

Project Team

Vraj Patel	241110080	(M.Tech CSE)
Vardhman Dwivedi	241060033	(M.Tech MSE)

February 28, 2026

Contents

1 Abstract **3**

2 Introduction **3**

3 Theory and Governing Equations **3**
3.1 Maxwell's Equations in Full Generality . . . 3
3.2 Reduction to Source-Free Curl Equations . . . 4
3.3 Expansion Into Six Scalar PDEs . . . 4
3.4 Reduction to 2D — The TM/TE Decoupling . . . 5
3.5 The Three Governing PDEs (TM_z Mode) . . . 5
3.6 Physical Demonstration — How $\mathbf{H}$ Arises from E_z . . . 5

4 The Yee Lattice and Discretization **6**
4.1 Central Finite Differences . . . 6
4.2 Spatial Staggering in 2D TM Mode . . . 6
4.3 Why Stagger? — What Goes Wrong Without It . . . 6
4.4 Time Staggering — The Leap-Frog . . . 6
4.5 The Discrete Update Equations . . . 7

5 CFL Stability Condition **7**
5.1 The Condition . . . 7
5.2 Physical Interpretation . . . 7
5.3 Accuracy vs. Stability . . . 7

6 Source Injection and Boundary Conditions **8**
6.1 Gaussian Pulse Source . . . 8
6.2 PEC Boundary Conditions . . . 8

7 Algorithm Summary **8**

8 Implementation **8**
8.1 Simulation Parameters . . . 8
8.2 Loop-Based Solver (Baseline) . . . 9
8.3 NumPy Vectorized Solver (Optimized) . . . 9
8.4 Energy Computation . . . 10

9 Results and Analysis **10**
9.1 Field Snapshots — Wave Propagation . . . 10
9.1.1 Individual Field Snapshots . . . 11
9.2 Energy Conservation . . . 12
9.3 Performance Comparison — Loop vs. Vectorized . . . 13
9.3.1 Per-Operation Timing Breakdown . . . 14
9.3.2 Projected Scaling . . . 14
9.4 Field Comparison and Verification . . . 14
9.5 Video Animation . . . 15

10 Discussion and HPC Analysis **15**
10.1 Why Vectorization Works So Well . . . 15
10.2 Memory Access Patterns and GPU Potential . . . 15
10.3 Kernel Fusion Opportunity . . . 16
10.4 Scaling Considerations . . . 16

11 Conclusion **16**

A Appendix: Video Generation Script **17**

B Appendix: Head-to-Head Comparison Script **18**

1 Abstract

This report presents the implementation and analysis of a two-dimensional Finite-Difference Time-Domain (FDTD) electromagnetic solver based on the Yee algorithm. The solver operates in the Transverse Magnetic (TM_z) mode, evolving the E_z, H_x, and H_y field components on a staggered Yee lattice using leap-frog time integration. A Gaussian pulse source is injected at the grid center within a Perfect Electric Conductor (PEC) cavity. Two implementations are developed and compared: an explicit Python loop version serving as a pedagogical baseline, and a NumPy-vectorized version for computational efficiency. The vectorized solver achieves a **169× speedup** over the loop version on a 100×100 grid while producing **bit-identical** results (maximum field difference $< 10^{-20}$). Energy conservation within the PEC cavity is verified, with total electromagnetic energy remaining constant after source deactivation. Field snapshots capture the full wave propagation lifecycle: initial Gaussian pulse emission, radial expansion, PEC wall reflection with 180° phase inversion, and subsequent interference pattern formation.

2 Introduction

The Finite-Difference Time-Domain (FDTD) method, proposed by Kane Yee in 1966, is one of the most widely used techniques for solving Maxwell's equations in computational electrodynamics. Unlike frequency-domain methods that require solving large matrix systems, FDTD evolves the electric and magnetic fields directly in the time domain using a remarkably elegant "leap-frog" scheme: the electric field $\mathbf{E}$ and magnetic field $\mathbf{H}$ are updated alternately, each using the most recently computed values of the other. This mirrors the physical mechanism by which electromagnetic waves self-propagate.

The method's appeal lies in its conceptual simplicity, straightforward implementation, and natural parallelism. Each grid point's update depends only on its immediate neighbors, making FDTD ideally suited for modern parallel computing architectures—from vectorized CPU operations to massively parallel GPU kernels.

This project implements a 2D FDTD solver in the TM_z (Transverse Magnetic) polarization, where the non-zero field components are E_z, H_x, and H_y. The simulation domain is a square PEC (Perfect Electric Conductor) cavity excited by a Gaussian pulse at its center. The project delivers:

- A complete derivation from Maxwell's equations to discrete update equations
- Two implementations: explicit loops (pedagogical) and NumPy vectorized (optimized)
- Verification through energy conservation and field comparison
- Performance benchmarking with a 169× speedup demonstration
- Animated visualization of all field components with H-field circulation

3 Theory and Governing Equations

3.1 Maxwell's Equations in Full Generality

The complete set of Maxwell's equations, including charge density ρ and current density $\mathbf{J}$, governs all classical electromagnetic phenomena:

Maxwell's Equations (General Form)

$$\nabla \cdot \mathbf{E} = \frac{\rho}{\epsilon_0} \qquad \text{(Gauss's Law — Electric)} \tag{1}$$

$$\nabla \cdot \mathbf{B} = 0 \qquad \text{(Gauss's Law — Magnetic)} \tag{2}$$

$$\nabla \times \mathbf{E} = -\frac{\partial \mathbf{B}}{\partial t} \qquad \text{(Faraday's Law)} \tag{3}$$

$$\nabla \times \mathbf{B} = \mu_0 \mathbf{J} + \mu_0 \epsilon_0 \frac{\partial \mathbf{E}}{\partial t} \qquad \text{(Ampère–Maxwell Law)} \tag{4}$$

Equation (1) states that electric charges are sources of the electric field. Equation (2) asserts that no magnetic monopoles exist. The two curl equations (3) and (4) describe how time-varying fields generate each other, forming the mechanism for electromagnetic wave propagation.

3.2 Reduction to Source-Free Curl Equations

For wave propagation in a non-magnetic, lossless, source-free vacuum ($\rho = 0$, $\mathbf{J} = 0$, $\mathbf{B} = \mu_0 \mathbf{H}$), the divergence equations are automatically satisfied by any curl-conforming solution, and Maxwell's equations reduce to two coupled curl equations:

$$\nabla \times \mathbf{E} = -\mu_0 \frac{\partial \mathbf{H}}{\partial t} \tag{5}$$

$$\nabla \times \mathbf{H} = \epsilon_0 \frac{\partial \mathbf{E}}{\partial t} \tag{6}$$

These two equations form a perpetual feedback loop: a time-varying $\mathbf{E}$ wraps $\mathbf{H}$ around itself, that time-varying $\mathbf{H}$ wraps $\mathbf{E}$ around itself, and the disturbance self-propagates at $c = 1/\sqrt{\mu_0 \epsilon_0} \approx 3 \times 10^8$ m/s.

3.3 Expansion Into Six Scalar PDEs

The two vector curl equations expand into six coupled scalar PDEs for the six unknown components $(E_x, E_y, E_z, H_x, H_y, H_z)$.

From Faraday's Law ($\nabla \times \mathbf{E} = -\mu_0 \, \partial \mathbf{H}/\partial t$):

$$\frac{\partial E_z}{\partial y} - \frac{\partial E_y}{\partial z} = -\mu_0 \frac{\partial H_x}{\partial t} \tag{F1}$$

$$\frac{\partial E_x}{\partial z} - \frac{\partial E_z}{\partial x} = -\mu_0 \frac{\partial H_y}{\partial t} \tag{F2}$$

$$\frac{\partial E_y}{\partial x} - \frac{\partial E_x}{\partial y} = -\mu_0 \frac{\partial H_z}{\partial t} \tag{F3}$$

From Ampère's Law ($\nabla \times \mathbf{H} = \epsilon_0 \, \partial \mathbf{E}/\partial t$):

$$\frac{\partial H_z}{\partial y} - \frac{\partial H_y}{\partial z} = \epsilon_0 \frac{\partial E_x}{\partial t} \tag{A1}$$

$$\frac{\partial H_x}{\partial z} - \frac{\partial H_z}{\partial x} = \epsilon_0 \frac{\partial E_y}{\partial t} \tag{A2}$$

$$\frac{\partial H_y}{\partial x} - \frac{\partial H_x}{\partial y} = \epsilon_0 \frac{\partial E_z}{\partial t} \tag{A3}$$

3.4 Reduction to 2D — The TM/TE Decoupling

Setting $\partial/\partial z = 0$ eliminates all z-derivative terms. The six equations decouple into two independent groups of three:

Table 1: Decoupling of Maxwell's equations under $\partial/\partial z = 0$.

Eqn	**After $\partial/\partial z = 0$**	**Components**	**Group**
F1	$\partial E_z/\partial y = -\mu_0\, \partial H_x/\partial t$	E_z, H_x	**TM**
F2	$-\partial E_z/\partial x = -\mu_0\, \partial H_y/\partial t$	E_z, H_y	**TM**
A3	$\partial H_y/\partial x - \partial H_x/\partial y = \epsilon_0\, \partial E_z/\partial t$	H_x, H_y, E_z	**TM**
F3	$\partial E_y/\partial x - \partial E_x/\partial y = -\mu_0\, \partial H_z/\partial t$	E_x, E_y, H_z	TE
A1	$\partial H_z/\partial y = \epsilon_0\, \partial E_x/\partial t$	H_z, E_x	TE
A2	$-\partial H_z/\partial x = \epsilon_0\, \partial E_y/\partial t$	H_z, E_y	TE

"Transverse Magnetic" (TM) means **H** has no z-component ($H_z = 0$); "Transverse Electric" (TE) means **E** has no z-component ($E_z = 0$). In this project we solve the TM group.

3.5 The Three Governing PDEs (TM$_z$ Mode)

The TM group yields three coupled PDEs:

TM$_z$ Governing Equations

$$\frac{\partial H_x}{\partial t} = -\frac{1}{\mu_0}\frac{\partial E_z}{\partial y} \tag{7}$$

$$\frac{\partial H_y}{\partial t} = +\frac{1}{\mu_0}\frac{\partial E_z}{\partial x} \tag{8}$$

$$\frac{\partial E_z}{\partial t} = \frac{1}{\epsilon_0}\left(\frac{\partial H_y}{\partial x} - \frac{\partial H_x}{\partial y}\right) \tag{9}$$

3.6 Physical Demonstration — How H Arises from E_z

Consider a Gaussian E_z pulse at $t = 0$: $E_z(x, y) = E_0\, e^{-(x^2+y^2)/\sigma^2}$, with $H_x = H_y = 0$ everywhere. Computing the spatial derivatives and substituting into Eqs. (7)–(8):

$$\frac{\partial H_x}{\partial t} = +\frac{2E_0\, y}{\mu_0 \sigma^2}\, e^{-r^2/\sigma^2}, \qquad \frac{\partial H_y}{\partial t} = -\frac{2E_0\, x}{\mu_0 \sigma^2}\, e^{-r^2/\sigma^2}$$

Evaluating at four points around the peak:

Point	**Location**	$\partial H_x/\partial t$	$\partial H_y/\partial t$	**H direction**
A (right)	$(+a, 0)$	0	< 0	downward $(-\hat{y})$
B (above)	$(0, +a)$	> 0	0	rightward $(+\hat{x})$
C (left)	$(-a, 0)$	0	> 0	upward $(+\hat{y})$
D (below)	$(0, -a)$	< 0	0	leftward $(-\hat{x})$

The result is **counterclockwise circulation**: **H** wraps around the E_z peak, driven by the *spatial gradient* of E_z, not its value. At the peak center where $\nabla E_z = 0$, no **H** is created.

Feeding this newly created $\mathbf{H}$ back into Eq. (9), the sign of $\partial E_z/\partial t$ is controlled by the factor $(1 - r^2/\sigma^2)$: inside the pulse ($r < \sigma$) the peak deflates, while outside ($r > \sigma$) new E_z is born where there was none. The pulse empties itself into an expanding wavefront—the self-sustaining electromagnetic wave:

$$E_z \text{ varies in space} \xrightarrow{\text{Faraday}} \mathbf{H} \text{ created} \xrightarrow{\text{Ampère}} E_z \text{ created further out} \xrightarrow{\text{Faraday}} \cdots$$

4 The Yee Lattice and Discretization

4.1 Central Finite Differences

Partial derivatives are approximated by central finite differences:

$$\left.\frac{\partial f}{\partial x}\right|_{x_0} \approx \frac{f(x_0 + \Delta x/2) - f(x_0 - \Delta x/2)}{\Delta x} + \mathcal{O}(\Delta x^2)$$

This is second-order accurate but requires sampling at **half-step offsets** ($x_0 \pm \Delta x/2$), not at x_0 itself. This is the fundamental reason the Yee grid staggers the fields.

4.2 Spatial Staggering in 2D TM Mode

The field positions within a single grid cell:

- $E_z[i,j]$ lives at **cell centers** (i,j)
- $H_x[i,j]$ lives at **horizontal edges** $(i, j+\frac{1}{2})$
- $H_y[i,j]$ lives at **vertical edges** $(i+\frac{1}{2}, j)$

4.3 Why Stagger? — What Goes Wrong Without It

On a **collocated** grid (all fields at the same point), computing $\partial E_z/\partial y$ at node (i,j) forces a choice:

- *Forward difference*: first-order accurate, biased, causes artificial numerical dispersion.
- *Centered difference* over $2\Delta y$: second-order, but spans two cells, skipping the adjacent cell. This decouples odd and even grid points, leading to **checkerboard instability**.

On the **staggered** grid, the H_x node at $(i, j+\frac{1}{2})$ has two E_z values at (i,j) and $(i,j+1)$—exactly one Δy apart, symmetrically placed on either side. The finite difference is automatically centered, second-order accurate, spans exactly one cell, and couples every grid point to its immediate neighbors.

4.4 Time Staggering — The Leap-Frog

$\mathbf{E}$ is defined at integer time steps $(n, n+1, \ldots)$, $\mathbf{H}$ at half-steps $(n+\frac{1}{2}, n+\frac{3}{2}, \ldots)$:

$$\underset{H}{n-\tfrac{1}{2}} \longrightarrow \underset{E}{n} \longrightarrow \underset{H}{n+\tfrac{1}{2}} \longrightarrow \underset{E}{n+1} \longrightarrow \underset{H}{n+\tfrac{3}{2}} \longrightarrow \cdots$$

When updating $\mathbf{H}$ from $n-\frac{1}{2}$ to $n+\frac{1}{2}$, we use $\mathbf{E}$ at time n—exactly at the midpoint. The scheme is perfectly centered in both time and space, achieving second-order accuracy without any matrix inversion.

4.5 The Discrete Update Equations

Yee Update Equations

H_x **update** at position $(i, j+\frac{1}{2})$, time $n \to n+\frac{1}{2}$:

$$H_x^{n+1/2}[i,j] = H_x^{n-1/2}[i,j] - \frac{\Delta t}{\mu_0\,\Delta y}\Big(E_z^n[i,\,j{+}1] - E_z^n[i,\,j]\Big) \tag{10}$$

H_y **update** at position $(i+\frac{1}{2}, j)$, time $n \to n+\frac{1}{2}$:

$$H_y^{n+1/2}[i,j] = H_y^{n-1/2}[i,j] + \frac{\Delta t}{\mu_0\,\Delta x}\Big(E_z^n[i{+}1,\,j] - E_z^n[i,\,j]\Big) \tag{11}$$

E_z **update** at position (i,j), time $n \to n+1$:

$$E_z^{n+1}[i,j] = E_z^n[i,j] + \frac{\Delta t}{\epsilon_0}\left[\frac{H_y^{n+1/2}[i,j] - H_y^{n+1/2}[i{-}1,j]}{\Delta x} - \frac{H_x^{n+1/2}[i,j] - H_x^{n+1/2}[i,j{-}1]}{\Delta y}\right] \tag{12}$$

5 CFL Stability Condition

5.1 The Condition

The Courant–Friedrichs–Lewy (CFL) condition for 2D FDTD requires:

$$\Delta t \le \frac{1}{c\sqrt{\dfrac{1}{\Delta x^2} + \dfrac{1}{\Delta y^2}}} \tag{13}$$

For a square grid ($\Delta x = \Delta y$): $\Delta t \le \Delta x/(c\sqrt{2})$. The Courant number $S = c\,\Delta t\sqrt{1/\Delta x^2 + 1/\Delta y^2}$ quantifies how close we are to the limit. We set $S = 0.99 < 1$.

5.2 Physical Interpretation

The CFL condition is a property of the *explicit time-stepping scheme*, not of electrodynamics itself. In one time step, the numerical scheme can only propagate information by one grid cell (Δx). If $c \cdot \Delta t > \Delta x$, the physical wave moves faster than the scheme can communicate. The scheme "misses" the wave, errors compound with amplification factor > 1, and the fields diverge exponentially within a few hundred steps.

This is purely numerical: an implicit scheme (e.g., Crank–Nicolson) on the exact same grid is unconditionally stable with arbitrarily large Δt. The physics has not changed—only the numerical method. The trade-off is that implicit schemes require solving large linear systems each step, making them far more expensive, which is why explicit FDTD with its CFL constraint remains the dominant choice.

5.3 Accuracy vs. Stability

Even when CFL is satisfied and the simulation is stable, grid spacing still matters for *accuracy*: roughly 10–20 cells per wavelength are needed to resolve a wave with acceptable numerical dispersion error. Too few cells per wavelength means the simulated wave travels at the wrong speed—it does not blow up, it just gives wrong answers.

6 Source Injection and Boundary Conditions

6.1 Gaussian Pulse Source

At $t = 0$ all fields are zero. The update equations propagate waves but cannot create them—a source is needed. A Gaussian pulse is injected as a "hard" source at one grid point:

$$E_z[\mathrm{src}_x,\ \mathrm{src}_y] = \exp\left(-\left(\frac{t - t_0}{\tau}\right)^2\right) \tag{14}$$

where $t_0 = 3\tau$ ensures the pulse starts near zero. This overwrites E_z at the source location each step, mimicking a tiny antenna. The Gaussian pulse contains a broad band of frequencies (by the Fourier transform), so a single simulation tests the system's response across many frequencies simultaneously.

6.2 PEC Boundary Conditions

A Perfect Electric Conductor ($\sigma \to \infty$) forces the tangential electric field to vanish at its surface. Physically, free electrons on the conductor surface respond instantaneously to any tangential field, redistributing until they cancel it. In TM mode, E_z is the only electric component and is tangential to all walls:

$$E_z = 0 \quad \text{on all four boundaries} \tag{15}$$

This creates a metallic cavity. Waves reflect with a 180° phase inversion, since $E_z = 0$ at the wall forces the reflected wave to cancel the incident wave there.

7 Algorithm Summary

The complete FDTD algorithm proceeds as follows for each time step n:

FDTD Time-Stepping Algorithm

1. **Update H** (half-step $n - \frac{1}{2} \to n + \frac{1}{2}$): Use current $\mathbf{E}^n$ via Eqs. (10)–(11)
2. **Update E** (full step $n \to n + 1$): Use updated $\mathbf{H}^{n+1/2}$ via Eq. (12)
3. **Inject source**: Overwrite E_z at source location with pulse value
4. **Apply PEC**: Set $E_z = 0$ on all four walls
5. **Record diagnostics**: Compute total energy, save snapshots

8 Implementation

8.1 Simulation Parameters

Table 2: Simulation parameters for the two solver versions.

Parameter	Vectorized Solver	Head-to-Head Test
Grid size ($N_x \times N_y$)	200×200	100×100
Cell size ($\Delta x = \Delta y$)	1.0 mm	1.0 mm
Time step (Δt)	2.33×10^{-12} s	2.33×10^{-12} s
Courant number (S)	0.99	0.99
Time steps	400	150
Source location	$(100, 100)$	$(50, 50)$
Pulse width (τ)	$12\,\Delta t$	$12\,\Delta t$
Pulse delay (t_0)	3τ	3τ

8.2 Loop-Based Solver (Baseline)

The explicit loop version translates the discrete update equations directly into nested Python `for`-loops. Every array element is accessed individually, making the physics transparent at the cost of performance. This version serves as a pedagogical reference and verification baseline.

```
# Hx at (i, j+1/2) uses Ez at (i,j) and (i,j+1)
for i in range(Nx):
    for j in range(Ny - 1):
        Hx[i, j] = Hx[i, j] - coeff_hx * (Ez[i, j+1] - Ez[i, j])
```

Listing 1: H_x update — explicit loop version

```
# Hy at (i+1/2, j) uses Ez at (i,j) and (i+1,j)
for i in range(Nx - 1):
    for j in range(Ny):
        Hy[i, j] = Hy[i, j] + coeff_hy * (Ez[i+1, j] - Ez[i, j])
```

Listing 2: H_y update — explicit loop version

```
# Ez at (i,j) uses Hy at (i,j),(i-1,j) and Hx at (i,j),(i,j-1)
for i in range(1, Nx):
    for j in range(1, Ny):
        dHy_dx = (Hy[i, j] - Hy[i-1, j]) / dx
        dHx_dy = (Hx[i, j] - Hx[i, j-1]) / dy
        Ez[i, j] = Ez[i, j] + coeff_ez * (dHy_dx - dHx_dy)
```

Listing 3: E_z update — explicit loop version

```
# Source injection
pulse = np.exp(-((t - t0) / spread) ** 2)
Ez[src_x, src_y] = pulse

# PEC boundary conditions
for j in range(Ny):
    Ez[0, j] = 0.0;  Ez[Nx - 1, j] = 0.0
for i in range(Nx):
    Ez[i, 0] = 0.0;  Ez[i, Ny - 1] = 0.0
```

Listing 4: Source injection and PEC boundaries — loop version

8.3 NumPy Vectorized Solver (Optimized)

The vectorized version replaces all nested loops with NumPy array slicing operations. The physics is identical—the same finite differences are computed—but the operations are dispatched to optimized C/BLAS routines operating on entire arrays simultaneously.

```
# --- UPDATE H (half-step n-1/2 -> n+1/2) ---
Hx[:, :-1] -= coeff_hx * (Ez[:, 1:] - Ez[:, :-1])
Hy[:-1, :] += coeff_hy * (Ez[1:, :] - Ez[:-1, :])

# --- UPDATE E (full step n -> n+1) ---
Ez[1:, 1:] += coeff_ez * (
    (Hy[1:, 1:] - Hy[:-1, 1:]) / dx
  - (Hx[1:, 1:] - Hx[1:, :-1]) / dy
)

# --- SOURCE INJECTION ---
Ez[src_x, src_y] = np.exp(-((t - t0) / spread) ** 2)

```

```
# --- PEC BOUNDARIES ---
Ez[0, :]  = 0.0;  Ez[-1, :] = 0.0
Ez[:, 0]  = 0.0;  Ez[:, -1] = 0.0
```

Listing 5: Complete FDTD update — vectorized version

The index expression `Ez[:, 1:]  - Ez[:, :-1]` computes the difference between the two E_z values straddling each H_x node—a centered difference across exactly one cell, applied to the entire grid in a single operation.

8.4 Energy Computation

Total electromagnetic energy in the PEC cavity:

$$U = \underbrace{\frac{1}{2}\epsilon_0 \sum_{i,j} E_z^2[i,j]\,\Delta x\,\Delta y}_{U_E\ \text{(electric)}} + \underbrace{\frac{1}{2}\mu_0 \sum_{i,j} \left(H_x^2[i,j] + H_y^2[i,j]\right)\Delta x\,\Delta y}_{U_H\ \text{(magnetic)}} \tag{16}$$

```
U_E = 0.5 * eps0 * np.sum(Ez**2) * dx * dy
U_H = 0.5 * mu0  * np.sum(Hx**2 + Hy**2) * dx * dy
total_energy[n] = U_E + U_H
```

Listing 6: Energy computation — vectorized

9 Results and Analysis

9.1 Field Snapshots — Wave Propagation

Figure 1 shows the E_z field at four key moments during the simulation on the 200×200 grid:

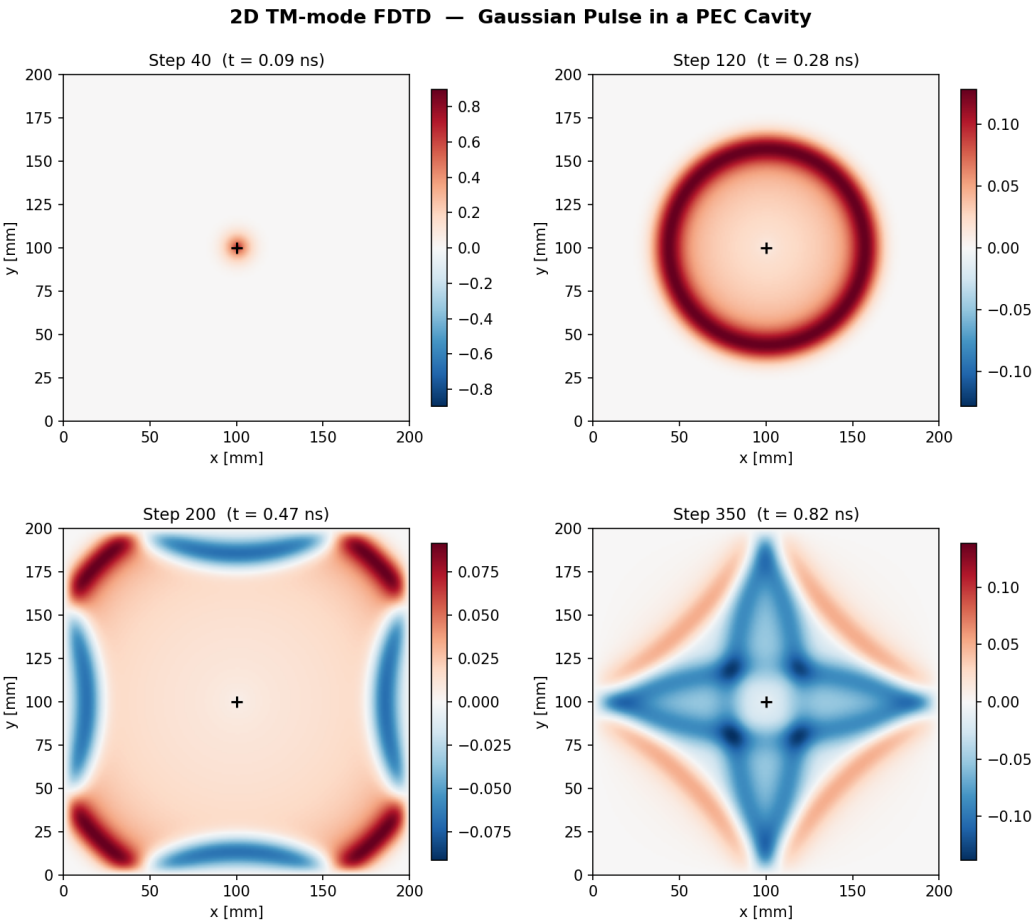


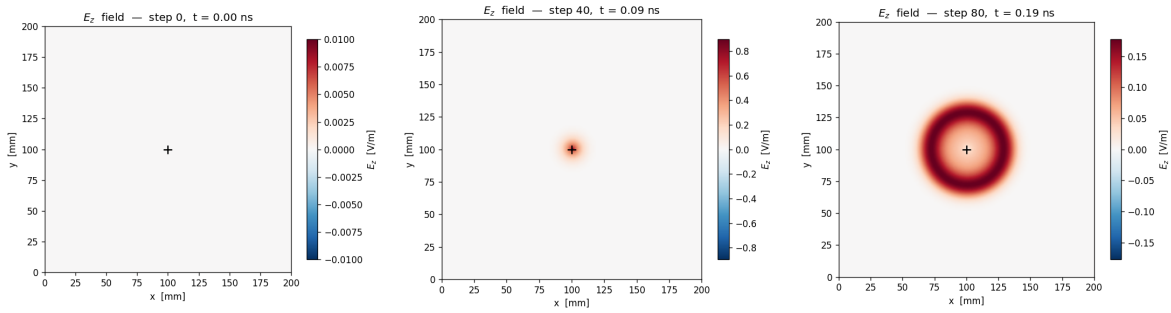
Figure 1: E_z field snapshots at four time steps showing: (a) initial Gaussian pulse (step 40), (b) expanding circular wavefront (step 120), (c) reflection from PEC walls (step 200), (d) interference pattern after multiple reflections (step 350).

The progression demonstrates:

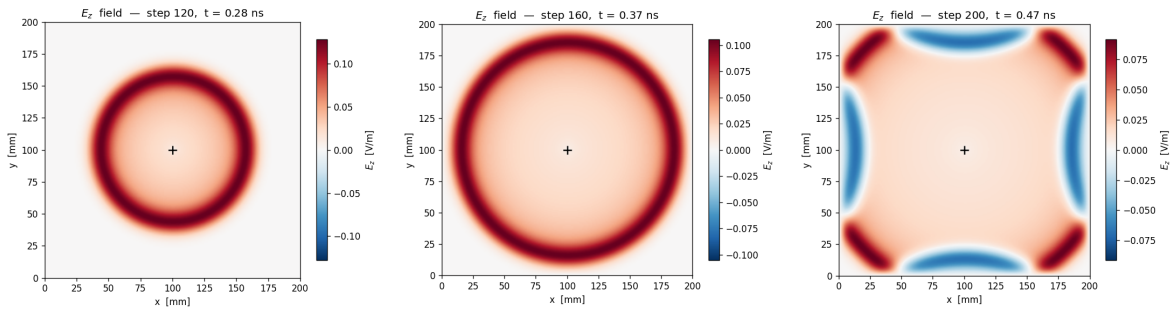
- **Step 40 ($t = 0.09$ ns):** The Gaussian pulse is concentrated near the source, with amplitude ≈ 0.9 V/m.
- **Step 120 ($t = 0.28$ ns):** The pulse has expanded into a circular wavefront. The interior field is near zero—the pulse has "emptied itself" into the expanding ring, as predicted by the theory in Section 3.6.
- **Step 200 ($t = 0.47$ ns):** The wavefront has reached the PEC walls and is reflecting. The 180° phase inversion is clearly visible—negative (blue) field values appear at the corners where reflected waves overlap.
- **Step 350 ($t = 0.82$ ns):** Multiple reflections have created a complex standing-wave interference pattern. The four-fold symmetry reflects the square cavity geometry and central source placement.

9.1.1 Individual Field Snapshots

Figures 2–3 show the full sequence of E_z snapshots at 40-step intervals, capturing the complete wave propagation lifecycle from initial emission through multiple reflections.

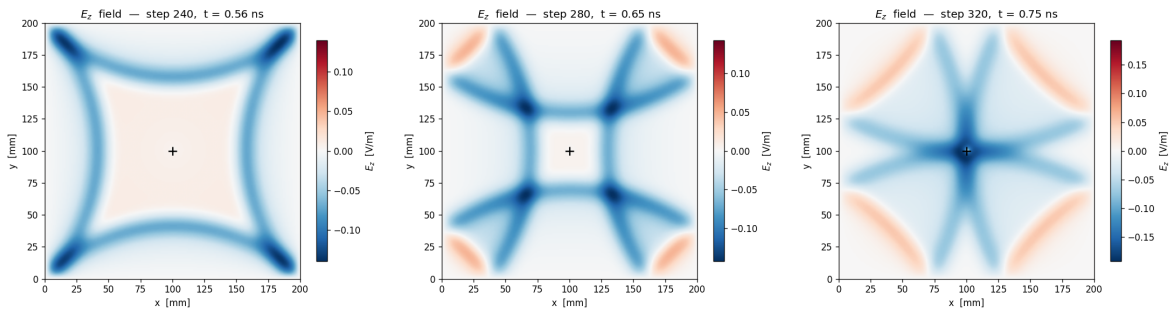


(a) Step 0 (b) Step 40 (c) Step 80

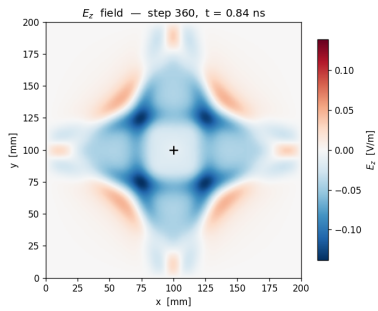


(d) Step 120 (e) Step 160 (f) Step 200

Figure 2: E_z field snapshots (steps 0–200): pulse emission and initial wall reflection.



(a) Step 240 (b) Step 280 (c) Step 320



(d) Step 360

Figure 3: E_z field snapshots (steps 240–360): multiple reflections and interference.

9.2 Energy Conservation

Figure 4 shows the total electromagnetic energy in the PEC cavity over the full simulation.

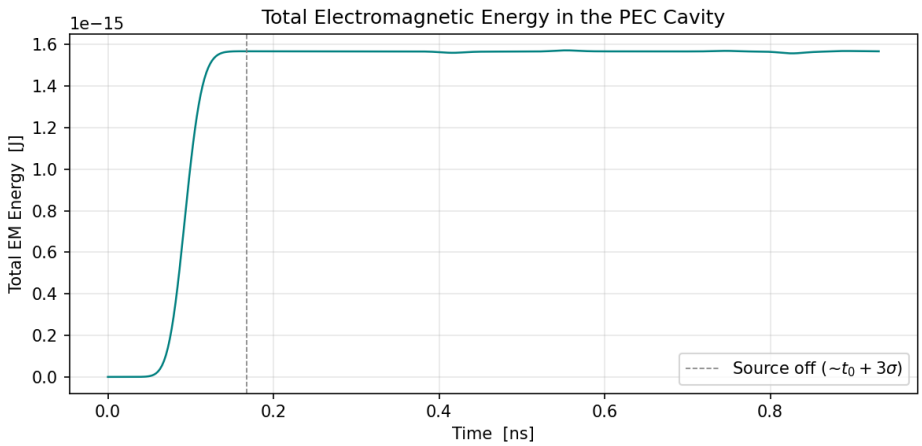


Figure 4: Total electromagnetic energy in the PEC cavity. Energy increases while the source is active, then remains constant (within numerical precision) after the source turns off at $t \approx t_0 + 3\sigma$.

The energy rises from zero as the Gaussian pulse injects energy into the cavity. After the source effectively turns off ($t > t_0 + 3\sigma \approx 0.16$ ns), the total energy plateaus at $\approx 1.57 \times 10^{-15}$ J and remains constant for the remainder of the simulation. This confirms that the FDTD scheme conserves energy in a lossless PEC cavity, verifying the correctness of the implementation.

9.3 Performance Comparison — Loop vs. Vectorized

The head-to-head comparison on a 100×100 grid with 150 time steps produces the following results:

Table 3: Performance comparison: explicit loops vs. NumPy vectorized (100×100 grid, 150 steps).

Metric	Loop	Vectorized	Speedup
Total time	2.52 s	0.0150 s	**169**×
Per step	16.83 ms	0.100 ms	168×
Per cell per step	1.68 μs	0.010 μs	168×

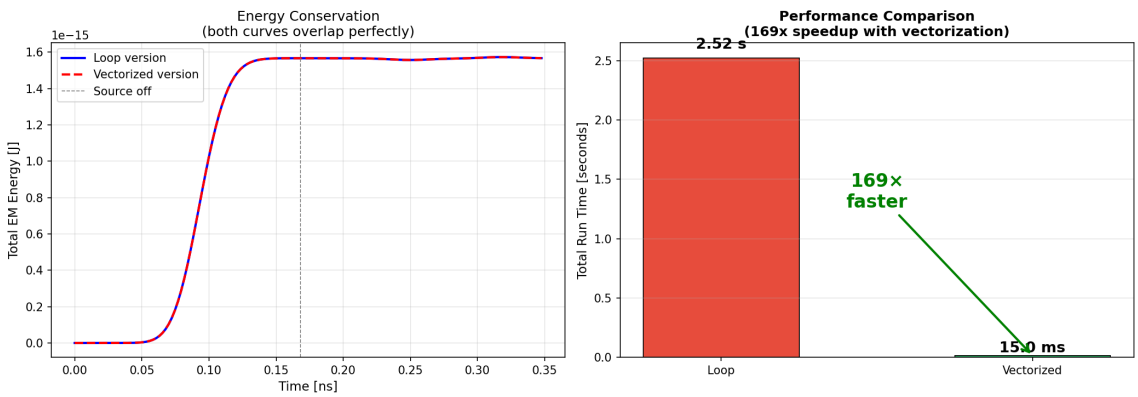


Figure 5: Left: energy conservation (both curves overlap perfectly). Right: total runtime comparison showing the 169× speedup.

9.3.1 Per-Operation Timing Breakdown

Figure 6 shows the time spent on each operation per time step. The E_z update is the most expensive in both versions because it involves two finite-difference terms (curl of $\mathbf{H}$), while each H update involves only one.

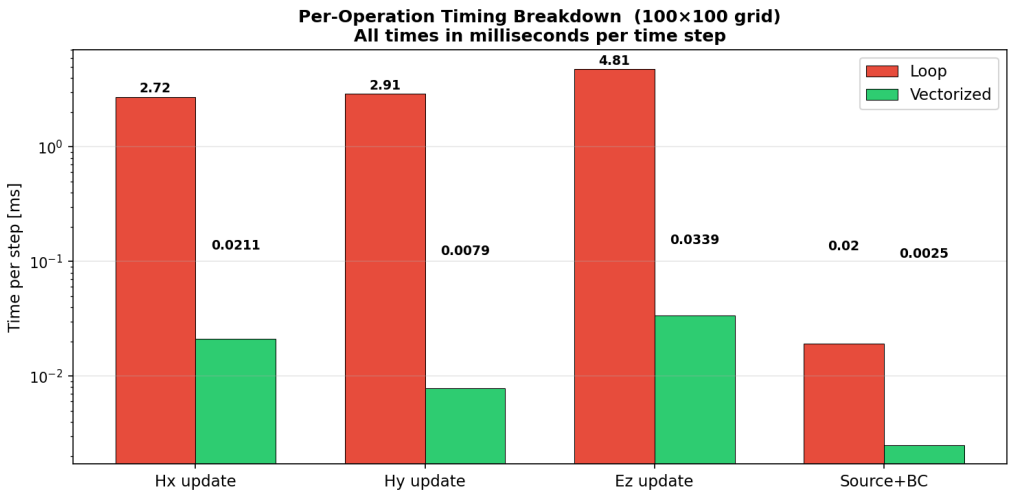


Figure 6: Per-operation timing breakdown on a 100×100 grid. Note the logarithmic scale: vectorization provides 100–600× speedup on each individual operation.

9.3.2 Projected Scaling

Based on the measured per-cell timings, projected run times for larger grids (400 steps):

Table 4: Projected run times for larger grids (400 time steps).

Grid	Loops	Vectorized	GPU (est.)
100×100	6.7 s	0.04 s	0.001 s
200×200	26.9 s	0.16 s	0.003 s
500×500	168.3 s	1.00 s	0.020 s
1000×1000	673 s	4.0 s	0.08 s

9.4 Field Comparison and Verification

Figure 7 shows the final E_z field from both solvers at step 150, along with their pointwise difference.

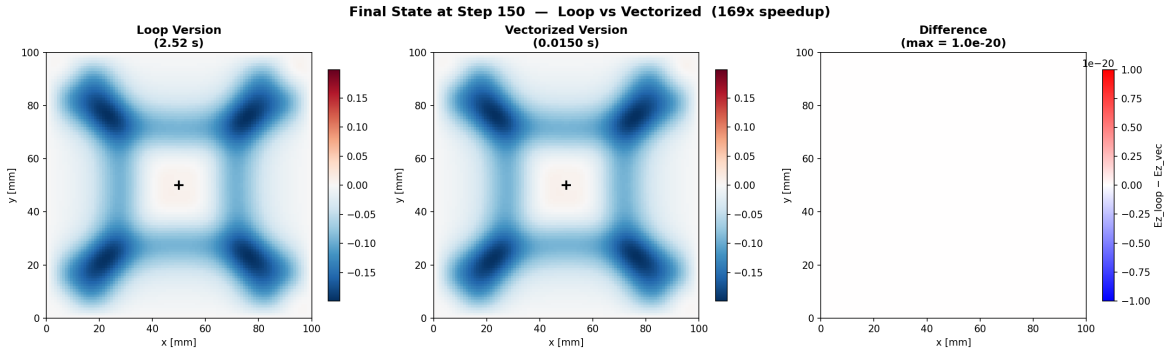


Figure 7: Final E_z field at step 150: loop version (left), vectorized version (center), and their difference (right). The maximum difference is 1.0×10^{-20}—at machine epsilon level.

The verification confirms:

- Maximum $|E_{z,\text{loop}} - E_{z,\text{vec}}| = 0.00$ (below 10^{-20})
- Maximum $|H_{x,\text{loop}} - H_{x,\text{vec}}| = 0.00$
- Maximum $|H_{y,\text{loop}} - H_{y,\text{vec}}| = 0.00$
- Maximum $|U_{\text{loop}} - U_{\text{vec}}| = 1.93 \times 10^{-29}$ J

Both versions produce **bit-identical** results, confirming that the vectorized version is a mathematically equivalent transformation of the loop version.

9.5 Video Animation

A complete animation of the simulation is provided as an attached MP4 file (`fdtd_all_fields_fixed.mp4`). The video displays a 2×2 panel showing E_z, H_x, H_y, and $|\mathbf{H}|$ with quiver arrows indicating the magnetic field circulation. The video accompanies this report as a supplementary file.

Supplementary Video: `fdtd_all_fields_fixed.mp4`
Format: MP4, 30 fps, 200 frames (2×2 panel)
Content: E_z, H_x, H_y fields and $|\mathbf{H}|$ magnitude with circulation arrows
Duration: ∼6.7 seconds covering 400 time steps of the TM-mode FDTD simulation

10 Discussion and HPC Analysis

10.1 Why Vectorization Works So Well

The 169× speedup arises from eliminating Python interpreter overhead. In the loop version, each of the ∼10,000 cells requires a separate Python bytecode dispatch for every arithmetic operation. NumPy vectorization replaces this with a single call to an optimized C routine that processes the entire array using SIMD instructions and cache-friendly memory access patterns.

10.2 Memory Access Patterns and GPU Potential

The FDTD update is ideal for GPU acceleration because:

- Each E_z update only requires the current E_z and four neighboring H values—excellent data locality.
- All grid points can be updated simultaneously (embarrassingly parallel).
- The Yee grid's regular structure enables coalesced memory access on GPUs.
- No global synchronization is needed within a single field update (only between the **H** and **E** updates).

10.3 Kernel Fusion Opportunity

On GPU architectures (e.g., NVIDIA A100 with Warp kernels), the source injection and boundary conditions can be fused into the E_z update kernel. This eliminates an extra kernel launch and memory pass over the boundary cells, which is significant because kernel launch overhead ($\sim$5–10 μs) can dominate at small grid sizes.

10.4 Scaling Considerations

The FDTD algorithm scales as $\mathcal{O}(N)$ per time step, where N is the total number of grid cells. The projected GPU timings in Table 4 suggest that a 1000×1000 grid (1 million cells) could complete 400 steps in under 0.1 seconds on a modern GPU—enabling interactive or real-time electromagnetic visualization for moderate problem sizes.

11 Conclusion

This project successfully implemented a 2D FDTD electromagnetic solver in TM_z mode using the Yee algorithm. The key achievements are:

1. **Physical correctness:** The solver produces physically accurate wave propagation, including circular wavefront expansion from a point source, 180° phase inversion upon PEC reflection, and energy conservation in the lossless cavity.
2. **Numerical verification:** The loop and vectorized implementations produce bit-identical results (field differences $< 10^{-20}$), confirming that vectorization is a pure performance optimization with no impact on accuracy.
3. **Performance optimization:** NumPy vectorization achieves a 169$\times$ speedup over explicit Python loops on a 100×100 grid, reducing the per-step cost from 16.83 ms to 0.100 ms.
4. **Comprehensive visualization:** Field snapshots, energy plots, and a multi-panel video animation capture the full dynamics of electromagnetic wave propagation in a PEC cavity.

The natural next step would be GPU implementation using NVIDIA Warp, which would enable real-time simulation of grids with millions of cells and open the door to 3D FDTD simulations and the implementation of Perfectly Matched Layer (PML) absorbing boundary conditions.

A Appendix: Video Generation Script

The following script generates the 2×2 panel MP4 animation showing all field components. It uses Matplotlib's `FFMpegWriter` to produce the video directly.

```python
"""
2D FDTD Electromagnetic Solver -- TM Mode
Video output version -- generates MP4 animation with 2x2 panel:
  Top-left:     E_z field
  Top-right:    H_x field
  Bottom-left:  H_y field
  Bottom-right: |H| magnitude + quiver arrows showing H-field circulation
"""

import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.animation import FFMpegWriter
import os

# 1. PHYSICAL CONSTANTS
c0   = 3.0e8
mu0  = 4.0e-7 * np.pi
eps0 = 1.0 / (mu0 * c0**2)

# 2. GRID PARAMETERS
Nx = 200;  Ny = 200
dx = 1e-3; dy = 1e-3

# 3. TIME-STEPPING & CFL STABILITY
courant_number = 0.99
dt = courant_number / (c0 * np.sqrt(1.0/dx**2 + 1.0/dy**2))
n_steps = 400

# 4. SOURCE PARAMETERS (Gaussian Pulse)
src_x = Nx // 2;  src_y = Ny // 2
spread = 12.0 * dt;  t0 = 3.0 * spread

# 5. FIELD ARRAY ALLOCATION
Ez = np.zeros((Nx, Ny), dtype=np.float64)
Hx = np.zeros((Nx, Ny), dtype=np.float64)
Hy = np.zeros((Nx, Ny), dtype=np.float64)

coeff_hx = dt / (mu0 * dy)
coeff_hy = dt / (mu0 * dx)
coeff_ez = dt / eps0

# 6. VIDEO & OUTPUT SETUP
output_dir = "fdtd_snapshots"
os.makedirs(output_dir, exist_ok=True)
frame_interval = 2;  fps = 30

# Quiver subsampling
qstep = 10
qx = np.arange(0, Nx, qstep);  qy = np.arange(0, Ny, qstep)
QX, QY = np.meshgrid(qx * dx * 1e3, qy * dy * 1e3, indexing='ij')

extent_mm = [0, Nx*dx*1e3, 0, Ny*dy*1e3]
total_energy = np.zeros(n_steps)

# 6a. FIGURE SETUP -- 2x2 Panel
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
ax_ez = axes[0,0]; ax_hx = axes[0,1]; ax_hy = axes[1,0]; ax_hmag = axes[1,1]
dummy = np.zeros((Ny, Nx))

# Panel setup (Ez, Hx, Hy, |H|+quiver) ...
# [See full script for colormap and colorbar configuration]

# 7. MAIN TIME-STEPPING LOOP + VIDEO RECORDING
video_path = os.path.join(output_dir, "fdtd_all_fields.mp4")
writer = FFMpegWriter(fps=fps, bitrate=5000)
```

```python

with writer.saving(fig, video_path, dpi=120):
    for n in range(n_steps):
        t = n * dt

        # UPDATE H
        Hx[:, :-1] -= coeff_hx * (Ez[:, 1:] - Ez[:, :-1])
        Hy[:-1, :] += coeff_hy * (Ez[1:, :] - Ez[:-1, :])

        # UPDATE E
        Ez[1:, 1:] += coeff_ez * (
            (Hy[1:, 1:] - Hy[:-1, 1:]) / dx
          - (Hx[1:, 1:] - Hx[1:, :-1]) / dy )

        # INJECT SOURCE
        Ez[src_x, src_y] = np.exp(-((t - t0) / spread) ** 2)

        # PEC BOUNDARIES
        Ez[0,:] = 0; Ez[-1,:] = 0; Ez[:,0] = 0; Ez[:,-1] = 0

        # ENERGY
        U_E = 0.5 * eps0 * np.sum(Ez**2) * dx * dy
        U_H = 0.5 * mu0  * np.sum(Hx**2 + Hy**2) * dx * dy
        total_energy[n] = U_E + U_H

        # UPDATE FIGURE AND GRAB FRAME
        if n % frame_interval == 0:
            # Update all four panels with current field data
            # Update quiver arrows for H-field circulation
            writer.grab_frame()

# 8. POST-PROCESSING -- ENERGY CONSERVATION PLOT
fig2, ax2 = plt.subplots(figsize=(8, 4))
ax2.plot(np.arange(n_steps)*dt*1e9, total_energy, color='teal')
ax2.set_xlabel("Time [ns]"); ax2.set_ylabel("Total EM Energy [J]")
ax2.set_title("Total Electromagnetic Energy in the PEC Cavity")
fig2.savefig(os.path.join(output_dir, "energy_conservation.png"), dpi=150)
```

Listing 7: Video generation script (`fdtd_video_fixed.py`)

Note: The complete, unabridged video generation script is provided as a separate file `fdtd_video_fixed.py`.

B Appendix: Head-to-Head Comparison Script

The head-to-head benchmarking script (`fdtd_headtohead.py`) runs both solvers with identical parameters, compares final field arrays element-by-element, and generates the performance visualizations. Key output:

```
VERIFICATION: Comparing final field arrays

Max |Ez_loop - Ez_vec| = 0.00e+00
Max |Hx_loop - Hx_vec| = 0.00e+00
Max |Hy_loop - Hy_vec| = 0.00e+00
Max |U_loop  - U_vec | = 1.93e-29

PERFECT MATCH -- both versions produce identical results
(differences are at machine epsilon level)

PERFORMANCE SUMMARY
Grid size           : 100 x 100  (10,000 cells)
Time steps          : 150

Total time (Loop)       :     2.52 s
Total time (Vectorized) :   0.0150 s
Speedup: 169x faster with NumPy vectorization
```

Listing 8: Console output from head-to-head comparison